

RMC Block Reference

Every Block in the v1.9.8 Architecture Diagram

46 blocks across 7 domains — name, function, key ports

AXI Masters

N_CLIENTS host masters issuing AXI4 transactions. Outside RMC's own scope — drawn as the traffic source.

(system-level, N_CLIENTS instances)

AXI Interconnect

Routes N_CLIENTS masters to the CIF's write/read ports. Outside RMC's scope.

routes to: AXI Write Port, AXI Read Port

AXI Write Port

Terminates the AXI4 write subordinate port. Valid-ready, spec-mandated.

`AWID/AWADDR/AWLEN/AWVALID → | AWREADY,WREADY,BVALID ←`

AXI Read Port

Terminates the AXI4 read subordinate port. Valid-ready, spec-mandated.

`ARID/ARADDR/ARLEN/ARVALID → | RDATA,RLAST,RVALID ←`

AMU (Address Map Unit)

Replaces the plain Address Translator. Per-field XOR hash + split-field extract, CSR-configurable setup time. Rank/ch hashed to MSBs (spread); bg/bank/row/col left raw (preserve row-hit locality).

`byte_addr → | ch,rank,bg,bank,row,col ←`

Burst Splitter

Stage 1: row-boundary check. Stage 2: BL16 alignment check.

`raw_addr,raw_len,raw_id → | split_addr[],split_valid ←`

ROB (Reorder Buffer)

Tags every request with composite {AXID,seqnum}; per-AXID head pointer re-orders DRAM-side out-of-order completions back into AXI's required per-ID order.

`alloc_id,alloc_seqnum → | rob_slot,rd_data_out ←`

Merge Logic

Reassembles any request the Burst Splitter fragmented.

`frag_data,frag_mask,frag_id → | merged_data,merged_valid ←`

Async REQ FIFO (CIF → MC)

The *only* CDC crossing in the whole design, request direction. CIF side: credit-based push, no combinational `wr_full`. MC side: valid-credit receive, registered `rd_valid`, credit returned after consume. `FIFO_DEPTH = 16`.

```
packet: req_type,axi_id,addr,data,mask | credit_return (MC→CIF, registered)
```

Async RESP FIFO (MC → CIF)

The reverse CDC crossing. MC side: credit-based push, gated by `gate_resp_fifo_avail = (local_credit > reserved_slots)` — MC never issues a read without this. CIF side: valid-credit receive, always accepts immediately.

```
packet: resp_type,axi_id,data,status | credit_return (CIF→MC, registered)
```

Write Data Buffer

Index-addressed SRAM — *not* a FIFO. Random access via `data_buf_idx` carried in the `WR_TCAM` entry.

`wr_idx,wr_data,wr_mask,wr_en → | rd_data,rd_mask ←`

WR_TCAM

Full address match `{bg,bank,row,col}` for exact-match RAW hazard detection. No valid/timestamp fields in-entry — gated externally by `status.valid`.

`raw_search_key → | raw_hit_vector,raw_hit_entry ←`
24,576T baseline

RD_TCAM

Ternary match `{bg,bank}` only — row/col carried, not matched. Feeds Scheduler Stage 1's per-bank pre-filter.

`sched_search_key → | sched_hit_bitmap,sched_hit_meta ←`
1,920T baseline

WR Status Reg

`valid|status|age` per `WR_TCAM` entry — single source of truth for occupancy. Owner: WR Watermark Mgr. Scheduler: READ ONLY.

`alloc_idx,update_idx,update_status → | rd_valid[],rd_age[] ←`

RD Status Reg

Same pattern + `merge_pending,wdb_entry_idx` for RAW partial-hit tracking. Owner: RD Watermark Mgr. Scheduler: READ ONLY.

`alloc_idx,update_status → | rd_valid[],rd_age[],merge_pending ←`

WR / RD Watermark Buffer Mgr

Own their TCAM+status-reg pair outright: alloc, retire, watermark tracking. Scheduler never writes either.

`new_req_valid,sched_ack → | wr/rd_count,high/low_wm_hit ←`

RAW Bypass Manager

Stage A: WR_TCAM exact-match search parallel with RD alloc (zero-penalty miss). Stage B: mask-coverage decides full-hit / partial-overlap / miss. Hit valid only if `wr_age <= rd_age`.

`new_rd_bg/bank/row/col,wr_tcam_hit_vector → | raw_hit,raw_data ←`

Merge Unit

64×2:1 byte mux, combinational. Fires only when `merge_pending==1`:
`merged[b] = wdb[b] if mask[b] else dram[b]`.

`wdb_data,wdb_mask,dram_data,dram_valid → | merged_data ←`

Hold-Forward (2-deep)

Holds up to 2 same-cycle responses: `src0` = RAW bypass, `src1` = DRAM return. A 3rd simultaneous collision is impossible by construction (each source rate-limited to 1/cycle).

`src0/i_valid,src0/i_data → | out_valid,out_data ←`

Global Cycle Counter

Free-running `GC_WIDTH` counter, resets only on `SOFT_RESET`. The timestamp source for every absolute-deadline field in the design.

`clk,rst_n → | gc ←` (read by all blocks)

timing_reg_file

param_id $\rightarrow$ nCK value lookup for every JEDEC timing parameter (tRCD, tRP, tRAS, tFAW, tRFC1/sb, tREFI...). Combinational multi-port read; CSR-only write at init.

param_id[] $\rightarrow$ | param_val[] $\leftarrow$ (combinational)

Bank Partition Ctrl

Splits banks into RD-half/WR-half, rotating every WINDOW_SIZE cycles
— eliminates tRTW/tWTR inside a window.

wr_count, WR_HIGH_WM $\rightarrow$ | rd/wr_partition_mask $\leftarrow$ S2

Write Data Path

WL alignment, CRC generation, DFI write-enable timing.

wr_data_buf_data/mask, CWL $\rightarrow$ | dfi_wrdata, dfi_wrdata_en $\leftarrow$

Read Data Path

Read-latency counter, capture FIFO, ECC/CRC check. MRR sideband output feeds MR_Poll FSM directly (not through the response FIFO).

dfi_rddata, dfi_rddata_valid $\rightarrow$ | rd_data_out, mrr_data_valid $\leftarrow$

Stage 0 — Maintenance Override

Priority $\text{ref_urgent} > \text{ref_due} > \text{rfm_req} > \text{zq_due}$. If any fires, bypasses Stages 1–3 straight to Stage 4.

$\text{ref_urgent}, \text{ref_due}, \text{zq_due}, \text{rfm_req} \rightarrow \mid \text{s0_override}, \text{s0_cmd_*} \leftarrow$

Stage 1 — TCAM Search

Searches RD_TCAM + WR_TCAM simultaneously (multi-port). Multi-hit same bank resolved by $\text{argmax}(\text{age})$.

$\text{wr_rd_tcam_hit_bitmap}, \text{meta} \rightarrow \mid \text{s1_hit_bitmap}, \text{s1_hit_meta} \leftarrow$

Stage 2 — can_* Gate + SJF Cost

Reads only registered can_* flags — zero subtractors in critical path. Classifies hit-set/miss-set with cost; detects speculative-ACT opportunities on NOP cycles.

$\text{can_cas/pre/act/ref}, \text{can_*_any/bg/rank}, \text{rd/wr_partition_mask} \rightarrow \mid$
 $\text{hit_set}, \text{remaining_cost}[] \leftarrow$

Stage 3 — SJF Winner Select

Priority: $\text{S0-override} > \text{starved-miss (staggered, } \leq 1/\text{cycle})} > \text{hit-set (prefer different BG)} > \text{miss-set (lowest cost)}$. RD/WR mode flips on watermark crossings.

$\text{hit/miss_set}, \text{remaining_cost}[], \text{rd/wr_status_age} \rightarrow \mid$
 $\text{winner_valid}, \text{winner_cmd_type} \leftarrow$

Stage 4 — Command Emission + Writeback

The *only* writer of Per-Bank/Global-Timing/Per-Rank tables and status registers. Drives DFI via the ME mux.

$\text{winner_*}, \text{s0_override}, \text{timing_reg_vals} \rightarrow \mid$
 $\text{dfi_*}, \text{bank_fsm_update_*}, \text{sched_ack} \leftarrow$

NOP Priority + Bank Pipeline (policy, not a stage)

NOP order: $\text{opportunity REFsb} > \text{speculative ACT} > \text{WR-partition drain} > \text{true NOP}$. Consecutive ACTs prefer different BG ($\text{tRRD_S} < \text{tRRD_L}$).

(folded into Stage 2/3 logic - no separate ports)

Per-Bank FSM Table

[N_RANKS×16] rows. 8-state bank FSM (IDLE..SELF_REFRESH), open row, next_{cas,pre,act,ref}+can_* pairs, ref_pending.

update_state,update_next_*,gc → | can_cas/pre/act/ref_out ← S2
writer: Stage 4 + ME • reader: Stage 2

Per-Rank FSM Table

[N_RANKS] rows. Rank FSM, REFab/ZQcal/PDX/SRX deadlines, leaky-bucket refresh credits, RAA[16], MR4 thermal state, last_refsb_gc[32].

set/clr_gate_rfc/zq,raa_inc_en → | gate_rfc/zq_out,can_rfc/zq_out
←
writer: ME • reader: ME + Stage 0

Global Timing Table

1 instance. Cross-bank tRRD_S/tCCD_S, tFAW ring buffer[4], per-BG tRRD_L/tCCD_L/tWTR_L family, per-rank tRRD/tCCD.

update_faw,update_act/cas_bg,gc → | can_faw_out,can_act/cas_bg_out
← S2
writer: Stage 4 • reader: Stage 2

Bank Activity Counter

[N_RANKS×16], count|dirty. ME targets least-loaded bank for REFsb; Power Mgmt uses count==0 as PD-entry gate; Stage 3 prefers high-count banks.

inc/dec_en,dirty_set/clr → | count_out,all_idle[rank] ←

Error Handler

Watches scheduler_error and dfi_alert_n (PHY CRC/CA-parity alert) plus Write/Read Data Path CRC/ECC flags.

scheduler_error,dfi_alert_n,crc/ecc_err → | err_interrupt,err_cmd
←

1. Init FSM (16 states)

DDR5 POR sequence: reset assert/deassert, ODT settle, DLL divider/reset (MPC), ZQ start/latch, MRW, training. Owns DFI until `init_done`.

`INIT_KICK, timing_reg_vals` → | `dfi_*, init_done` ← (one-way latch)

2. Refresh FSM (6 states)

Leaky-bucket credits (+1/tREFI, -1/REFsb). `ref_urgent` at 8 credits.

Targets least-loaded bank unless watchdog forces overdue bank. Fires free opportunity REFsb on NOP cycles.

`bank_act_count, all_idle` → | `ref_urgent, ref_due, me_cmd_*` ←

3. ZQcal FSM (7 states, per-rank)

MPC ZQ-calibration start→latch sequence. Gates the entire rank for the duration.

`all_idle, sched_ack` → | `zq_due, set/clr_gate_zq` ←

4. RFM FSM (6 states)

Tracks per-bank RAA (+1/ACT, -RAADec/REF). Requests targeted RFM once RAA crosses RAAIMT. Priority below refresh, above ZQcal.

`raa_out, RAAIMT` → | `rfm_req, raa_dec_en` ←

5. Power Mgmt FSM (10 states)

Independent PD and SR branches. PD entry requires every bank in the rank idle (`count==0`) with no maintenance pending.

`all_idle, bank_act_count, pd_en` → | `me_cmd_*, rank_state_update` ←

6. MR_Poll FSM (6 states)

Periodic MRR to MR4 reads thermal update flag. TUF=1 (>85°C) halves effective tREFI. Response via Read Data Path sideband, not resp FIFO.

`mrr_data_valid, mrr_data` → | `tuf_bit, me_cmd_valid` ←

DFI Output Mux

Lives inside the ME. While `init_done==0`, Init FSM drives every DFI output. Once `init_done` fires (one-way latch, never de-asserted), Scheduler Stage 4 drives DFI permanently. MRR issue is a Stage-0 bypass straight to Stage 4 — no third mux input ever needed.

```
init_dfi_*, sched_dfi_*, init_done → | dfi_address,dfi_cs_n,dfi_wrdata,...← PHY
```

Internal ME priority mirrors Scheduler Stage 0 exactly: $ref_urgent > ref_due > rfm_req > zq_due$. The Maintenance Engine never issues a CAS command itself — it only ever writes into the Per-Bank and Per-Rank FSM Tables, and requests Stage-0 override slots from the Scheduler for anything that must actually reach DFI.

DDR PHY

DFI 5.2 contract boundary. Handles IO calibration, DQS alignment, ODT, and the generation-specific electrical signalling RMC never has to know about.

`dfi_address,dfi_cs_n,dfi_wrdata_en → | dfi_rddata,dfi_rddata_valid,dfi_alert_n ←`

DDR5 DRAM

JESD79-5 device(s) the whole controller ultimately exists to drive correctly. Outside RMC's scope; drawn as the pipeline's final destination.

(JEDEC JESD79-5 physical/electrical interface, via PHY)